7. 확장 실험 및 향후 연구 방향

본 연구는 원곡 재현(§3-§4)을 넘어 여러 방향의 확장 실험을 수행하였다. 완료된 실험과 향후 과제를 함께 정리한다.

7.1 모듈 단위 생성 + 구조적 재배치 (Most Immediate)

본 연구는 지금까지 "전체 $T=1{,}088$ timesteps에 해당하는 음악을 한 번에 생성"하는 방식을 사용했다. 그러나 §5 Figure 7에서 드러났듯이, hibari의 실제 구조는 **32-timestep 모듈이 33회 반복**되는 형태이며, 두 악기는 inst 1 = 연속적 기저 흐름, inst 2 = 모듈마다 쉼을 두는 얹힘 구조라는 상보적 역할을 가진다.

이 관찰을 반영한 새로운 생성 전략이 가능하다:

1. **한 모듈만 생성** — persistent homology로 얻은 cycle 집합 중 모듈 1개(32 timesteps)의 위상 구조를 보존하는 최소 단위의 생성을 먼저 수행한다. 이는 기존 파이프라인을 $T=32$로 줄여 그대로 실행한 뒤, 중첩행렬의 한 모듈 폭 만큼만 seed로 사용하면 된다.
2. **hibari 구조에 따라 배치** — 생성된 모듈을 33회 반복하되:

- inst 1 자리에는 동일한 모듈을 쉼 없이 이어 붙임

- inst 2 자리에는 같은 모듈을 각 반복마다 inst 1 대비 shift (Figure 7 관찰에 맞게) + 모듈마다 지정된 위치에 쉼을 삽입

3. **모듈 간 이행 매끄럽게** — 단순 복제가 아니라 모듈 경계에서 자연스러운 이행이 이루어지도록 ±2 timesteps 정도의 "이음새" 영역을 가변 cycle로 재샘플링.

이 접근의 이점은 다음과 같다.

- **계산 효율**: $T=1{,}088$ 대신 $T=32$로 persistent homology를 계산하므로 수 배 빠르다.
- **구조적 충실도**: hibari의 고유 모듈 구조를 재샘플링이 아닌 동일한 모듈의 복제로 보존하므로 곡의 정체성이 유지된다.
- **변주 가능성**: 한 모듈의 cycle seed만 바꾸면 전체 곡 수준의 변주가 자동으로 만들어진다.

7.2 다른 곡으로의 일반화 — solari 실험 결과

본 연구의 §3.4 해석 8("hibari의 FC 우위는 곡의 미학적 성격에서 기인")을 검증하기 위해, 같은 out of noise 앨범의 **solari**에 동일한 파이프라인을 적용하였다. solari는 hibari와 같은 앨범에 수록되어 있으나, 12개 pitch class를 모두 사용하는 반음계적(chromatic) 곡으로 성격이 대비된다. 실험 러너는 `run_solari.py`, 결과는 `docs/step3_data/solari_results.json`.

solari 기본 정보: hibari와 동일한 GCD 기반 tie 정규화(pitch-only labeling)를 적용하여 $N=34$ 고유 (pitch, duration) 쌍, $T=224$ timesteps, 49개 고유 화음, tempo ≈ 29 BPM.

Algorithm 1 — 거리 함수 비교

거리 함수	cycle 수	density	JS (mean ± std)	JS min
frequency	22	0.070	0.063 ± 0.005	0.056
Tonnetz	39	0.037	0.063 ± 0.003	0.059
voice-leading	25	0.043	0.078 ± 0.004	0.073

hibari에서는 Tonnetz가 frequency 대비 47% 우위였지만, solari에서는 **frequency와 Tonnetz가 거의 동일** (0.063 vs 0.063)하며 voice-leading이 가장 나쁘다. 이는 §3.6에서 분석한 solari의 12-PC 구조 — Tonnetz 그래프의 지름이 2에 불과하여 구별력이 낮음 — 과 일치한다.

Algorithm 2 — DL 모델 비교

설정	FC	LSTM	Transformer
binary JS	0.106	0.168	0.032
continuous JS	0.042	0.171	0.016

핵심 발견: solari에서는 Transformer가 최적. hibari와 정확히 반대 패턴이다. hibari에서 FC 최적 / Transformer 열등이었던 것이, solari에서는 Transformer가 FC의 2.6배 ($0.042 \to 0.016$, continuous 기준) 우위이다.

곡의 성격이 최적 모델을 결정한다는 가설을 지지. hibari (diatonic, entropy 0.974, 공간적 배치)에서는 시간 문맥을 무시하는 FC가 최적이었고, solari (chromatic, entropy 0.905, 선율적 진행)에서는 시간 문맥을 적극 활용하는 Transformer가 최적이다. 이 대비는 §4.4 해석 8을 실증적으로 뒷받침한다.

곡	PC 수	정규화 entropy	최적 거리	최적 모델	해석
hibari	7 (diatonic)	0.974	Tonnetz	FC	공간적 배치, 시간 무관
solari	12 (chromatic)	0.905	frequency/Tonnetz 동등	Transformer	선율적 진행, 시간 의존

Continuous overlap의 효과

solari에서도 continuous overlap은 이진 대비 개선을 보였다. Transformer 기준 binary JS $0.032 \to$ continuous JS 0.016 (50% 감소). 이 개선폭은 hibari의 개선 F (57% 감소)와 비슷한 수준으로, continuous overlap의 효과가 곡의 특성에 독립적임을 시사한다.

향후 과제

대조군으로 **전통적 선율 인과가 강한 곡** (바흐 fugue, Ravel "Pavane" 등)에 같은 파이프라인을 적용하여 Transformer 우위가 더 강해지는지 검증이 필요하다. 이 두 그룹의 대비가 명확히 드러나면, "곡의 미학과 모델 구조의 정합성" 가설이 genre-dependent music modeling이라는 더 일반적인 framework로 확장될 수 있다.

7.3 방향 A: 거리 보존 note 재분배

지금까지의 모든 실험은 원곡의 pitch 분포를 가능한 한 재현하는 것을 목표로 했다. 본 절부터는 방향을 전환하여, **위상 구조를 보존하면서 원곡과 다른 음악**을 생성하는 문제를 다룬다.

아이디어

중첩행렬(시간 구조)은 그대로 보존하되, 각 cycle에 배정된 note를 새로운 note로 교체한다. 교체의 제약 조건은 다음과 같다:

1. 새 note 집합의 Tonnetz 거리행렬이 원본과 **up to permutation으로 유사**해야 한다 (정의 2.11)
2. 새 note 집합으로 구성한 거리행렬의 **persistence diagram이 원본과 유사**해야 한다 — Wasserstein distance (§2.10)로 측정
3. 새 note의 pitch 범위를 제어하여 얼마나 다른 곡을 원하는지를 조절한다

코드: `note_reassign.py`, 결과: `note_reassign_results.json`, `note_reassign_wasserstein_results.json`.

실험 설계

Tonnetz 거리 기반으로 랜덤 후보 1000개를 생성하고 2단계 최적화를 적용한다. 1단계: 비용 $0.5 \times \text{note_err} + 0.5 \times \text{cycle_err}$ 기준 상위 30개 후보 수집. 2단계: 상위 후보 각각에 대해 persistent homology를 재계산하고 원본 PD와의 Wasserstein distance를 추가 비용으로 더하여 최종 선택. pitch 범위를 변화시켜 "얼마나 다른 음"을 허용할지를 조절한다.

Algorithm 1 결과

설정	pitch 범위	note 오차	cycle 오차	DTW vs 원곡	pitch JS
baseline (원곡)	52-81	—	—	2.21	0.016
tonnetz₩_narrow	55-79	4.00	1.17	2.09 (-5.8%)	0.283
tonnetz₩_wide	48-84	4.46	1.18	2.89 ($+30.3\%$)	0.358

tonnetz₩_vwide	40-88	4.48	1.06	3.57 $(+61.4\%)$	0.419

해석: pitch 범위를 넓힐수록 DTW (선율 차이)가 크게 증가하여 vwide에서 $+61.4\%$에 달한다. 동시에 cycle 거리 오차는 ~ 1.1로 안정적 — 즉 **위상적 거리 구조는 보존되면서 선율은 확실히 다른 곡이 생성**된다.

Wasserstein distance 제약 결과

Tonnetz 매칭 + Persistence Diagram Wasserstein distance를 결합한 결과 (pitch 범위 40-88 고정):

설정	note 오차	cycle 오차	Wass. dist	pJS	DTW
제약 없음	4.35	1.35	—	0.607	4.13
Wass $\alpha = 0.3$	4.48	1.42	1.77	0.513	4.01
scale₩_major + Wass 0.5	3.52	1.48	1.06	0.115	2.78

해석: Wasserstein 제약을 추가하면 PD 보존이 개선되면서(Wass. 1.77) pJS도 $0.607 \rightarrow 0.513$으로 16% 감소한다. scale₩_major와 결합하면 Wass. 1.06으로 더 낮아지면서 pJS 0.115, DTW 2.78로 가장 균형 잡힌 결과이다.

Algorithm 2 (DL) 결과

재분배된 note 위에서 LSTM/Transformer를 재학습시킨 결과. **FC는 제외**하였는데, FC는 시점별 독립(i.i.d.) 모델이라 시간 맥락을 학습할 수 없어 재분배된 note의 시퀀스 패턴을 학습하기에 부적합하기 때문이다. 테이블 컬럼 의미:

- **vs 원곡 pJS**: 생성곡의 pitch 분포를 원곡과 비교한 JS divergence (다를수록 높음)
- **vs 원곡 DTW**: 생성곡과 원곡의 선율 차이 (다를수록 높음)
- **vs ref pJS**: 생성곡의 pitch 분포를 재분배된 note 분포(reference)와 비교한 JS (낮을수록 학습 정확)
- **cycle 오차**: 원곡과 새 note의 cycle-cycle Tonnetz 거리 차이

설정	모델	vs 원곡 pJS	vs 원곡 DTW	vs ref pJS	cycle 오차
baseline	Transformer	0.016	1.84	—	—
wide	LSTM	0.498	2.31	0.317	1.18
wide	Transformer	0.362	2.26	0.011	1.18
vwide	LSTM	0.441	2.01	0.317	1.06
vwide	Transformer	0.426	3.09	0.006	1.06

핵심 발견: Transformer는 재분배된 note의 분포를 거의 완벽하게 학습한다 (vs ref pJS $0.006 \sim 0.011$). 이는 Transformer가 중첩행렬 + 새 note 매핑을 정확히 재현할 수 있음을 의미하며, vwide + Transformer 조합에서 DTW 3.09 $(+68\%)$로 원곡과 가장 다른 선율이 생성된다.

7.4 방향 B: 중첩행렬 시간 재배치

방향 A가 어떤 note를 바꾸는 것이라면, 방향 B는 언제 연주하는지를 바꾸는 것이다. 중첩행렬의 **행(시점)**을 재배치하여 같은 cycle 구조를 다른 시간 순서로 전개한다.

3가지 재배치 전략

1. **segment_shuffle**: 전체 T를 길이 64의 세그먼트로 나누고, 세그먼트 순서를 무작위 셔플. 세그먼트 내부 순서는 유지.
2. **block_permute** (block size 32/64): 고정 크기 블록을 무작위 순열로 재배치.

3. **markov_resample** ($\tau = 1.0$): 원본 중첩행렬의 전이확률로부터 Markov chain을 추정하고, 온도 τ로 새 시퀀스를 생성 (정의 2.14). 여기서 **온도(temperature)** τ는 전이확률의 날카로움을 조절하는 파라미터이다. $\tau = 1$이면 원본 전이확률을 그대로 사용, $\tau > 1$이면 확률이 균등화되어 더 무작위적인 시퀀스가 생성되고, $\tau < 1$이면 확률이 날카로워져 가장 빈번한 전이만 반복된다.

코드: `temporal_reorder.py`, 결과: `temporal_reorder_dl_results.json`, `temporal_reorder_dl_v2_results.json`.

왜 Transformer만 사용하는가: 본 실험에서 FC와 LSTM은 제외하였다. FC는 시점별 독립 모델이므로 시간 재배치의 영향을 전혀 받지 않는다 (입력 순서가 바뀌어도 출력이 같다). LSTM은 내재적 시간 구조(recurrence)를 가지지만, positional embedding이 없으므로 PE 제거 실험의 대상이 아니다. Transformer만이 **명시적 positional embedding(PE)**을 사용하여 시간 위치 정보를 학습하므로, PE 유무에 따른 시간 재배치 효과를 비교하기에 가장 적합하다.

Transformer 결과 (PE 있음, 원본 학습)

전략	pitch JS	transition JS	DTW	DTW 변화
baseline	0.006	0.102	1.79	—
segment₩_shuffle	0.006	0.123	1.90	$+5.8\%$
block32	0.006	0.104	1.86	$+3.6\%$
markov ($\tau = 1.0$)	0.008	0.104	1.84	$+2.7\%$

재배치 효과가 미약하다. segment₩_shuffle이 가장 큰 변화를 보이지만 DTW $+5.8\%$에 불과하다.

PE 제거 + 재학습 실험

Transformer에서 positional embedding(PE)을 제거하고, 재배치된 중첩행렬로 재학습:

설정	pitch JS	transition JS	DTW	DTW 변화
noPE₩_baseline	0.011	0.128	1.85	—
noPE₩_segment₩_shuffle	0.011	0.149	1.87	$+1.0\%$
noPE+retrain segment₩_shuffle ★	0.173	0.399	2.22	$+21.7\%$
noPE+retrain markov ($\tau = 1.0$)	0.185	0.443	2.16	$+18.0\%$

noPE + markov (retrain 없음)를 진행하지 않은 이유. noPE₩_segment₩_shuffle (retrain 없음)에서 이미 $+1.0\%$에 불과한 변화만 관찰되었다. 이는 "retraining 없이 PE만 제거한 모델에 재배치된 입력을 넣으면 출력이 거의 바뀌지 않는다"는 패턴이 재배치 전략과 무관하게 성립함을 보인다. markov 재배치 역시 retrain 없이는 같은 패턴을 따를 것이므로 별도 실험을 추가하지 않았다.

방향 B의 딜레마: PE 제거 + 재학습에서 DTW가 $+21.7\%$까지 증가하여 선율이 확실히 바뀌지만, 동시에 pitch JS가 $0.007 \rightarrow 0.173$으로 **분포가 붕괴**된다.

- 약한 재배치 → pitch 보존, 선율 변화 없음
- 강한 재배치 → 선율 변화, pitch 분포도 붕괴

이 딜레마는 방향 B 단독으로는 "pitch 유지 + 선율 변화"를 동시에 달성하기 어려움을 의미한다. **방향 A (note 교체)와 결합**해야 양쪽의 장점을 취할 수 있다.

7.5 화성 제약 조건

방향 A의 note 재분배는 위상적 거리만 보존하므로, 결과가 **음악적으로 불협화**할 수 있다. 본 절은 화성(harmony) 제약을 추가하여 재분배의 음악적 품질을 개선한다.

3가지 화성 제약 방법

1. **scale 제약**: 새 note의 pitch class를 특정 스케일 (major, minor, pentatonic)에 한정. 허용 pool 크기가 줄어들지만 음악적 일관성이 보장된다.
2. **consonance 목적함수**: 재분배 비용에 평균 dissonance (정의 2.13)를 penalty로 추가. $\text{cost} = \alpha \cdot \text{dist_err} + \beta \cdot \text{diss}$.
3. **interval structure 보존**: 원곡의 interval class vector (정의 2.12)와 새 note 집합의 ICV 차이를 penalty로 추가.

결과: `note_reassign_harmony_results.json`.

Algorithm 1 결과

설정	note 오차	cycle 오차	평균 dissonance	scale match	PC 수
baseline (제약 없음)	4.35	1.35	0.412	0.70 (C♯ major)	10
scale_major ★	3.52	1.48	0.361	1.00 (C major)	7
scale₩_minor	3.68	1.69	0.363	1.00 (E major)	7
scale₩_penta	4.32	1.48	0.213	1.00 (C major)	5
consonance 단독	4.35	1.35	0.412	0.70	10
interval 단독	4.35	1.34	0.437	0.64	11

핵심 발견:

1. **scale_major가 가장 효과적**: note 거리 오차가 baseline $4.35 \to 3.52$로 19% 감소하면서 동시에 scale match 1.00, dissonance $0.412 \to 0.361$로 화성적 품질도 개선된다. C major로 고정되므로 원곡(hibari)의 조성과 일치한다.
2. **scale_penta가 가장 낮은 dissonance**: 0.213으로 baseline의 절반 수준. 5음만 사용하므로 불협화 자체가 구조적으로 억제된다.
3. **consonance 단독은 무효과**: 비용에 추가해도 최적 후보가 바뀌지 않았다. dissonance penalty가 거리 보존 제약에 비해 너무 약한 것으로 추정된다.

Algorithm 2 (Transformer) 결과

(§7.3의 DL 실험에서 Transformer가 LSTM보다 우수했으므로, 화성 제약 실험에서는 **Transformer만** 적용하였다.)

설정	vs 원곡 pJS	vs 원곡 DTW	vs ref pJS	val₩_loss
original	0.009	1.80	—	0.524
baseline (제약 없음)	0.600	3.46	0.007	0.497
scale_major ★	0.097	2.35	0.003	0.492
scale₩_penta	0.259	3.37	0.009	0.487

scale_major + Transformer 조합은 원곡 대비 pJS 0.097 (적당히 다름), DTW 2.35 ($+31\%$, 다른 선율), ref 대비 pJS 0.003 (재분배된 note 분포를 거의 완벽 학습)으로, **위상 보존 + 적당한 차이 + 화성적 일관성**의 균형이 가장 좋다.

7.6 A+B 결합 + continuous overlap — 최종 통합 실험

방향 A (note 재분배 + 화성 제약)와 방향 B (시간 재배치)를 결합하고, continuous overlap matrix를 적용한 최종 실험이다. 결과: `combined_AB_results.json`, `harmony_continuous_results.json`.

실험 설계

- **note 설정**: 원곡(orig) 또는 scale₩_major 재분배(major)
- **시간 설정**: none, segment₩_shuffle, block32, block64, markov ($\tau = 1.0$)
- **overlap**: binary 또는 continuous
- **모델**: Transformer (continuous overlap 직접 입력)

결합 실험 주요 결과

설정	note	reorder	vs 원곡 pJS	vs 원곡 DTW	vs ref pJS
orig₩_none	orig	none	0.014	1.52	0.014
orig₩_segment₩_shuffle	orig	shuffle	0.007	1.87	0.007
orig₩_block32	orig	block32	0.005	1.84	0.005
major_none	major	none	0.125	2.38	0.010
major_segment_shuffle	major	shuffle	0.117	2.45	0.012
major_block32 ★	major	block32	0.100	2.37	0.002
major₩_markov	major	markov	0.112	2.47	0.005

Continuous overlap 비교

설정	overlap	vs 원곡 pJS	vs 원곡 DTW	vs ref pJS	val₩_loss
orig₩_binary	binary	0.010	1.80	0.010	0.531
orig₩_continuous	continuous	0.012	1.51	0.012	0.102
major₩_binary	binary	0.100	2.31	0.009	0.524
major_continuous ★	continuous	0.125	2.34	0.010	0.107

Continuous overlap은 val₩_loss를 $0.53 \rightarrow 0.10$ (5배 감소)으로 대폭 낮춘다. DTW에서는 binary가 약간 더 높지만, 학습 품질(val₩_loss)에서 continuous가 압도적으로 우수하다.

최종 종합 — 가장 균형 잡힌 결과

평가 축	목표	major₩_block32 결과	판정
vs ref pJS (재분배 학습 정확도)	낮을수록 좋음	0.002	★ 최우수
vs 원곡 DTW (선율 차이)	높을수록 좋음	2.37 ($+31\%$)	충분히 다름
vs 원곡 pJS (pitch 차이)	적당히 높음	0.100	적당한 차이
scale match	1.0	1.00 (C major)	완전 일치

major_block32: scale₩_major 재분배 + block₩_permute(32) 시간 재배치 + continuous overlap + Transformer. 이 설정은:

- 원곡과 **같은 조성(C major)**을 유지하면서
- 위상적 거리 구조를 보존하고
- 선율은 DTW 기준 $+31\%$ 다르며
- Transformer가 재분배된 note 분포를 pJS 0.002로 거의 완벽하게 학습

major_segment_shuffle은 DTW가 가장 높지만 (2.45), ref 대비 pJS도 0.012로 학습 정확도가 약간 떨어진다. **major_markov**는 DTW 2.47로 가장 큰 선율 차이를 보이나 ref pJS 0.005.

이로써 본 연구는 "원곡과 위상수학적으로 유사하면서 음악적으로 다른 곡"을 생성할 수 있는 완전한 파이프라인을 제시한다.

7.7 Continuous overlap의 정교화

§3.3a에서 continuous overlap $\rightarrow$ $\tau = 0.5$ 이진화가 11.4% 개선을 주었지만, 이는 단일 고정 임계값만을 탐색한 것이다. 더 정교한 방향:

1. **Per-cycle 임계값.** 모든 cycle에 동일한 τ를 쓰지 않고, cycle 별로 고유 τ_c를 학습 가능 파라미터로 두어 $\{\tau_c\}_{c=1}^{K}$를 end-to-end optimization으로 결정한다.
2. **Soft activation을 받아들이는 Algorithm 2 변형.** 현재 Algorithm 2는 $O[t,c]$를 이진 입력으로 가정하는데, continuous $O_{\text{cont}}[t,c] \in [0,1]$을 그대로 받아들이는 입력 레이어로 교체한다. 이 경우 모델은 "어느 cycle이 얼마나 강하게 활성인가"라는 더 풍부한 정보를 학습에 쓸 수 있다.
3. **가중치 함수 학습.** 현재 희귀도 가중치 $w(n) = 1/N_{\text{cyc}}(n)$은 고정된 휴리스틱인데, 이를 학습 가능 함수로 대체하여 어떤 가중치가 JS divergence 최소화에 가장 도움이 되는지를 직접 추정한다.

7.8 Tonnetz Hybrid의 α grid search

본 연구는 Tonnetz hybrid의 α를 0.5 고정으로 실험했다. 다른 α 값 ($0.0, 0.1, 0.3, 0.5, 0.7, 1.0$) 에 대해 동일 $N = 20$ 반복 실험을 수행하면, "빈도 거리와 음악이론적 거리의 최적 혼합 비율"을 정량적으로 제시할 수 있다. 메모리에 있는 과거 단일 run 실험에서 $\alpha = 0.3$이 가장 좋았다는 힌트가 있으나, 통계적 확인이 필요하다.

7.9 Interactive 작곡 도구

현재 파이프라인의 모든 단계는 batch 처리 방식이다. 실시간/상호작용 버전은 다음과 같이 설계할 수 있다:

- 사용자가 GUI에서 **drag-and-drop으로 중첩행렬을 직접 그리면**, 그 중첩행렬로부터 Algorithm 1 또는 2가 즉시 음악을 생성하여 재생
- 사용자가 특정 cycle을 강조/억제하는 슬라이더를 조작하면 생성 결과가 실시간으로 바뀜
- Figure 2의 interactive HTML을 기반으로 확장하여, cycle을 클릭하면 그 cycle만 활성화된 "단색" 생성을 들을 수 있는 모드 추가

이 방향은 본 연구의 결과를 **작곡 보조 도구**로 활용하게 만든다. 전통적으로 TDA는 분석 도구로만 쓰여왔지만, 본 연구의 파이프라인은 "위상 구조를 직접 조작하여 음악을 만드는" 생성 도구로도 기능할 수 있음을 보였다.

7.10 위상 구조 보존의 수학적 엄밀화

본 연구의 "보존도 함수" $f(S) = 0.5J + 0.3C + 0.2B$ (§2.7) 는 경험적 선택이다. 향후 과제로는 다음이 있다:

1. **Submodularity 증명 시도.** 보존도 함수가 submodular라면 Nemhauser 정리에 의해 greedy 근사가 0.632-optimal임이 보장된다. 세 구성 요소 중 어떤 것이 submodular인지 분석하고, 비-submodular 요소가 있다면 그것을 대체하거나 penalty로 처리한다.
2. **Bottleneck/Wasserstein distance.** 보존도의 수학적 자연스러운 정의는 두 persistence diagram 사이의 bottleneck 또는 Wasserstein distance이다. 본 연구의 Jaccard/Correlation/Betti 조합을 이러한 이론적 기반과 연결하는 것이 과제다.

7.11 우선순위 정리

과제	상태	핵심 결과	우선순위
7.1 모듈 단위 생성	**완료**	P4+C JS 0.0258 (full-song 능가)	—
7.2 solari 일반화	**완료**	Transformer 최적 확인, 가설 확증	—
7.3 방향 A (note 재분배)	**완료**	vwide DTW $+61.4\%$, 거리 구조 보존	—
7.4 방향 B (시간 재배치)	**완료**	단독은 한계, A와 결합 필요	—
7.5 화성 제약	**완료**	scale₩_major 가장 효과적	—
7.6 A+B 결합	**완료**	major₩_block32 pJS 0.002, DTW $+31\%$	—
7.7 Continuous overlap 정교화	미완	per-cycle τ, soft activation 학습	높음
7.8 Tonnetz α grid search	미완	기존 결과의 robustness 확인	중
7.9 Interactive 작곡 도구	미완	연구 성과의 응용 확장	중
7.10 보존도 함수의 이론적 정당화	미완	이론적 기여	장기